United States Patent Application Publication	20250284765
Kind Code	A1
Publication Date	September 11, 2025
Inventor(s)	Boblest; Sebastian et al.

Method and Device for Efficiently Operating a Neural Network with Convolutions

Abstract

A method is for parallelized calculation of two convolutions of a filter having first and second receptive fields of the filter on input data. The first and second receptive fields correspond to a filter shifted by one step on the input data. The method includes initializing first and second output variables each having an initial value, and executing a loop for each line of the filter. The loop performs loading a first kernel element of the filter of the line and the corresponding data values to the first kernel element of the first and second receptive fields of the input data.

Inventors: **Boblest; Sebastian (Duernau, DE), Wagner; Benjamin (Friedrichshafen, DE), Khoi Vo; Duy (Stuttgart, DE), Sulaiman; Leif (Lund, SE), Lochmann; Markus (Reutlingen, DE), Hjort; Ulrik (Malmö, SE)**

Applicant: **Robert Bosch GmbH** (Stuttgart, DE)

Family ID: **1000008474175**

Appl. No.: **19/057659**

Filed: **February 19, 2025**

Foreign Application Priority Data

DE	10 2024 202 064.8	Mar. 05, 2024

Publication Classification

Int. Cl.: **G06F17/15** (20060101)

U.S. Cl.:

CPC **G06F17/15** (20130101);

Background/Summary

[0001] This application claims priority under 35 U.S.C. § 119 to patent application no. DE 10 2024 202 064.8, filed on Mar. 5, 2024 in Germany, the disclosure of which is incorporated herein by reference in its entirety.

[0002] The disclosure relates to a method for parallelized calculation of two convolutional operations of a filter, a device, a computer program and a machine-readable storage medium.

BACKGROUND

[0003] Convolutions are one of the most important layers for neural networks. In networks with convolutions (CNNs), these layers typically comprise over 90% of the total time it takes to calculate an inference. Efficient implementations of these are therefore very important and thus an object of this disclosure, in particular an implementation of certain convolutions that are common in modern CNNs, which increases data reuse and thus allows more efficient calculation on microcontrollers.

[0004] For more powerful architectures such as GPUs and CPUs, Winograd implementations are very often used that reduce the number of multiplications but use a large amount of RAM. Thus, for hardware platforms with limited RAM, these implementations cannot be used. Instead, a partial Im2Col implementation is used, see for example: K. Chellapilla, S. Puri, and P. Simard. High performance convolutional neural networks for document processing. Tenth International Workshop on Frontiers in Handwriting Recognition, University of Rennes 1, October 2006, La Baule (France), 2006.

SUMMARY

[0005] The disclosure addresses a shortcoming of Im2Col implementation in certain convolution configurations that are very commonly used in modern CNNs. Provided that the dilation is 1 and the stride is smaller than the kernel size, successive lines of the Im2Col buffer contain data in duplicate. At the same time, the data required for calculating an output pixel is already stored line by line in the memory. This can now be used to calculate two output pixels of the convolution without loading duplicate data and thus saving memory bandwidth.

[0006] In a first aspect, the disclosure relates to a computer-implemented method for parallelized calculation of two convolutions of a filter having a first and second receptive field of the filter on input data on a data processing system, in particular a microcontroller, with limited computing resources. The filter and the input data are at least two-dimensional (height, width and a number of input channels). A receptive field of a filter in a mathematical convolution refers to the range of the input data to which the filter is applied. It is the range where the filter applies the weights to generate the output. The receptive field may be considered a type of “field of view” of the filter. The receptive field is determined by the size of the filter and the step width (stride). The size of the filter specifies how many input values the filter considers simultaneously, while the step size specifies how far the filter is moved over the input data at each step. Limiting computing resources may relate to the following aspects: With respect to the CPU, in particular how many calculations and operations can be performed, and with respect to a memory/space or bandwidth.

[0007] The method starts with initializing first and second output variables (o.sub.000,o.sub.010) each having an initial value (b.sub.0), wherein the output variables (o.sub.000,o.sub.010) each have a result of the convolution of the filter with the first and second receptive fields. The initial value (b.sub.0) is typically a trained parameter of the neural network.

[0008] This is followed by the execution of a loop for each line r with a value range from 0 to the maximum number of filter coefficients of the filter along the dimension of its lines minus the value one. That is, the loop is executed over each line of the kernel matrix of the filter, wherein the loop performs the following steps: [0009] loading a first kernel element k.sub.r0c of the filter of the line

r and the corresponding data values d.sub.r0c, d.sub.r1c to the first kernel element (k.sub.r0c) of the first and second receptive fields of the input data. The corresponding data values d.sub.r0c d.sub.r1c to the first kernel element (k.sub.r0c) are those data points of the input data to which the first kernel element (k.sub.r0c) is to be applied according to the first and second receptive fields.
[0010] calculating the value of the first output variable (o.sub.000) by adding a product of d.sub.r0c and k.sub.r0c to a current value of the first output variable (o.sub.000), and calculating the value of the second output variable (o.sub.010) by adding a product of d.sub.r1c and k.sub.r0c to a current value of the second output variable (o.sub.010). [0011] loading the next kernel element (k.sub.r1c) of the filter from the corresponding line of the current loop index r. The next kernel element k.sub.r1c is the kernel element of the filter immediately following the first kernel element k.sub.r0c. [0012] calculating the value of the first output variable (o.sub.000) by adding a product of d.sub.r1c and k.sub.r1c to the current value of the first output variable (o.sub.000). [0013] loading further data values d.sub.r2c and d.sub.r3c. The further data values d.sub.r2c and d.sub.r3c are the data values already loaded d.sub.r0c, d.sub.r1c subsequent data values d.sub.r2c and d.sub.r3c of the first and second receptive fields, in particular the data values for which the next convolution operation is to be executed with the two following kernel elements k.sub.r1c and k.sub.r2c. [0014] calculating the value of the second output variable (o.sub.010) by adding the product of d.sub.r2c and k.sub.r1c to the current value of the second output variable (o.sub.010). loading the next kernel element k.sub.r2c. [0015] calculating the value of the first output variable (o.sub.000) by adding the product of d.sub.r2c and k.sub.r2c to a current value of the first output variable (o.sub.000), and calculating the value of the second output variable (o.sub.010) by adding the product of d.sub.r3c and k.sub.r2c to a current value of the second output variable (o.sub.010).
[0016] It should be noted that the method was exemplified for a filter having 3 kernel elements per line. In the event that the filter has 2 or more kernel elements per line, the method can be adjusted accordingly by loading (and calculating) more or fewer kernel elements together with corresponding data values correspondingly in alternating fashion.
[0017] Optionally, outputting the second (and/or first) output variable (o.sub.000, o.sub.010) may be performed as convolution results for the two convolutions.
[0018] After the method has been performed, it may be performed again for the next receptive field of the filter. In particular, the method is performed until the convolution of the filter with the entire input data has been performed. That is to say, until the input data received has been effectively covered by the receptive field.
[0019] It is proposed that the filter and the input data have a plurality of channels and another loop is executed for each channel c across the loop over the lines r. The filter and the input data may accordingly each be given as a tensor, preferably with the same number of channels. If the number of channels does not match, the method according to the disclosure can additionally be repeated along the dimension of the channels several times until the receptive field has effectively covered the entire tensor of the input data.
[0020] Furthermore, it is proposed that the kernel elements are stored according to a sequence k.sub.000; k.sub.010; k.sub.020; ::: k.sub.100; k.sub.110; k.sub.120; :::. That is to say, the kernel elements were read out line by line and the lines were compiled one after the other to form the sequence in question. This has the advantage that no pointer is required for the sequence of filter elements and only two pointers are needed for efficient processing of the input data.
[0021] In further aspects, the disclosure relates to a device and to a computer program, which are each configured so as to carry out the aforementioned methods, and to a machine-readable storage medium on which said computer program is stored.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

[0022] Embodiments of the disclosure are explained in greater detail below with reference to the accompanying drawings. In the drawings:

[0023] FIG. **1** schematically shows a representation of a partial Im2Col algorithm for a single output pixel for a convolution with (3×3) kernels without dilation with stride 1;

[0024] FIG. **2** schematically shows a representation of the partial Im2Col algorithm for two adjacent output pixels for a convolution with (3×3) kernels without dilation with stride 1;

[0025] FIG. **3** schematically shows an illustration for nested implementation of a (3×3) convolution with stride 1; and

[0026] FIG. **4** schematically shows pseudocode of an exemplary embodiment of the disclosure.

DETAILED DESCRIPTION

[0027] Partial Im2Col implementation is known from the prior art. For example, 2D convolutions are implemented in this way in the CMSIS-NN library (L. Lai, N. Suda, and V. Chandra. Cmsis-nn: Efficient neural network kernels for arm cortex-m cpus) from ARM.

[0028] FIG. **1** illustrates the procedure for calculating an output pixel (**10**) for a convolution (**11**) having a kernel size (3×3) and stride 1 and dilation 1 according to Im2Col implementation. The input data (**1***a*,**1***d*), which are preferably each present as 2D input data, such as a matrix, is to be convoluted with one or more filters (**2**).

[0029] The data required for one pixel in the output channels is copied sequentially from the input data to another storage area (**12**). The actual convolution is then a simple scalar product of the values in the kernel with those from the input data. The kernel is already in the right order. In order to use data, in particular kernel elements, several times, the data for several pixels is usually loaded from the input data. In addition, kernel elements are also loaded for multiple output channels (**13**).

[0030] How many pixels are loaded in parallel is a balance between the RAM required for this and the saving of duplicate loading processes for kernel elements and input data. In addition, the number of simultaneously loaded data is also limited by the available registers. A battery variable is required for each output pixel to be calculated in addition to the loaded data, so that the register demand increases very quickly. CMSIS NN, for example, loads the data for two pixels and two output channels, and then only needs to load the kernel data once for these two pixels. Four accumulator variables are then required. This number is adapted to the 13 registers of the Cortex-M architecture.

[0031] The use of Im2Col has the advantage that for each kernel size and each value for stride and dilation, the convolution is attributed to a matrix multiplication, which is also very efficient on CPUs compared to a nested loop implementation, which requires the input data to load multiple times.

[0032] In the following, a shortage of Im2Col implementation is to be addressed for certain convolution configurations that are very commonly used in modern CNNs. Provided that the dilation is 1 and the stride is smaller than the kernel size, successive lines of the Im2Col buffer contain data in duplicate, see FIG. **2**. That is to say, if the receptive field of the filter (**2**) is moved by one step, in FIG. **2** by one position horizontally to the right, for the next convolution after the convolution has been completed, then some of the data from the input data that was already in the receptive field of the filter in the previous step is reloaded into the memory area (**12**). As the receptive field has moved by one step to the right, only the three superimposed data points to the right of the previous data points in the memory area (**12**) are really new in the next step. The schematic first and second lines of the memory area (**12**) thus contain 66% of the data in duplicate without this being exploited in the Im2Col implementation. Instead, this data is loaded multiple times from its original location in memory into the memory area (**12**). At the same time, the data required for calculating an output pixel is already stored line by line in the memory, see FIG. **3**.

[0033] The index notation d.sub.ijk in FIG. **3** refers to the ith row, jth column and kth input

channel. This can now be used to calculate two output pixels of the convolution without loading duplicate data and thus saving memory bandwidth. The highlighted data to the left is used for calculating the output pixels 0 and 1. For each output pixel, the data is already stored line by line in memory. For example, for output pixels o.sub.000, the data d.sub.000 to d.sub.c03 in the first line.
[0034] FIG. **4** shows an exemplary computational specification in the form of a pseudo algorithm for resolving said defect. The structure of this calculation takes into account the properties of microcontrollers:
Data is not loaded until it is needed for the next calculation. This takes into account the fact that small microcontrollers have only a few registers in which variables are stored on the processor. Each kernel element is loaded only once for two output pixels. Each data value is loaded only once for two output pixels. The specific implementation may be adapted to Cortex-M architecture. For other architectures with more registers, 3 or more pixels may be calculated in parallel, which further increases data reuse.
[0035] This pseudocode represents a procedure called “Interleaved Convolution 3×3, Stride 1”. It performs a convolution operation on a 3×3 kernel with a step of 1. This is an exemplary embodiment, for example the filter dimension, etc., can be changed as desired.
[0036] The method of FIG. **4** begins with initializing output variables o.sub.000 and o.sub.010 with the value b.sub.0, which is zero, for example.
[0037] A loop is then started for each input channel c from 0 up to the total number of input channels. If only one channel is present, this loop may be left open.
[0038] Thereafter, a nested loop follows for each line r from 0 to 2, or up to the maximum number of filter coefficients of the filter (**2**) minus the value one. In the nested loop, the following steps are performed: [0039] loading the data values d.sub.r0c, d.sub.r1c and the kernel element k.sub.r0c.
[0040] calculating the value of o.sub.000 by adding a product of d.sub.r0c and k.sub.r0c to a current value of o.sub.000 and calculating the value of o.sub.010 by adding a product of d.sub.r1c and k.sub.r0c to a current value of o.sub.010. [0041] loading the next kernel element k.sub.r1c.
[0042] calculating the value of o.sub.000 by adding a product of d.sub.r1c and k.sub.r1c to the current value o.sub.000. [0043] loading data values d.sub.r2c and d.sub.r3c. [0044] calculating the value of o.sub.010 by adding the product of d.sub.r2c and k.sub.r1c to the current value o.sub.010.
[0045] loading the next kernel element k.sub.r2c. [0046] calculating the value of o.sub.000 by adding the product of d.sub.r2c and k.sub.r2c to the current value of o.sub.000 and calculating the value of o.sub.010 by adding the product of d.sub.r3c and k.sub.r2c to the current value of o.sub.010.
[0047] After the final step within the nested loop, either the nested loop for the next higher value for is r performed, or if r has reached its maximum value, the nested loop is terminated.
[0048] What has just been said regarding the termination of the nested loop applies analogously to the outer loop regarding the channels c.
[0049] After the procedure according to FIG. **4** has been performed, it may be performed again for the next receptive fields of the filter.
[0050] The embodiment described in FIG. **4** does not access the input data sequentially. For micro-controllers, this does not lead to run-time disadvantages. However, the machine code requires two pointers to run efficiently through the input data, which can limit the available registers. The same applies to the kernel elements. However, as the kernel elements are constant and are already known at the time of code generation, they can be reordered so that they are sequentially stored in memory. Instead of the old sequence k.sub.000; k.sub.001; :::; k.sub.010; k.sub.011; :::, the data is placed in the order k.sub.000; k.sub.010; k.sub.020; ::: k.sub.100; k.sub.110; k.sub.120; :::.
[0051] Microcontrollers with a DSP unit, such as Cortex-M4 and Cortex-M7, can execute two MAC instructions in parallel for quantized networks and can also load 4 bytes in parallel. For such architectures, the algorithm of FIG. **4** is modified so that two input channels are run in parallel. Correspondingly, the kernel reordering is then carried out with the elements from two channels in a

row, e.g. k.sub.000; k.sub.001; k.sub.010; k.sub.011; k.sub.020; k.sub.021; :::.
[0052] Note that convolutions are also calculated in int8 networks with int16 variables to avoid overflows, i.e. two variables correspond to 4 bytes.
[0053] The implementation has been described in the case of (3×3) kernels with stride 1 and dilation 1. Among the kernel configurations commonly used in modern CNNs, this is the configuration that benefits the most. However, analogously, it can also be implemented for (3×3) cores with stride 2 and for (2×2) cores with stride 1 and also for exotic kernel configurations in which data is used multiple times. For each of the cases just mentioned, a different reordering of the kernel elements results accordingly.

Claims

1. A method for parallelized calculation of two convolutions of a filter having first and second receptive fields on input data, the first and second receptive fields correspond to the filter shifted by one step on the input data, the method comprising: initializing first and second output variables (o.sub.000,o.sub.010) each having an initial value (b.sub.0), wherein the output variables (o.sub.000,o.sub.010) each have a result of a convolution of the filter with the first and second receptive fields; executing a loop for each line r of the filter, the loop performing: loading a first kernel element (k.sub.r0c) of the filter of the line r and corresponding data values d.sub.r0c, d.sub.r1c to the first kernel element k.sub.r0c of the first and second receptive fields of the input data; calculating a value of the first output variable (o.sub.000) by adding a product of d.sub.r0c and k.sub.r0c to a current value of the first output variable (o.sub.000), and calculating a value of the second output variable (o.sub.010) by adding a product of d.sub.r1c and k.sub.r0c to a current value of the second output variable (o.sub.010); loading a kernel element (k.sub.r0c) following the first kernel element (k.sub.r1c) of the filter of the line r; calculating the value of the first output variable (o.sub.000) by adding a product of d.sub.r1c and k.sub.r1c to the current value of the first output variable (o.sub.000); loading the data values d.sub.r2c and d.sub.r3c following the data values d.sub.r0c, d.sub.r1c of the first and second receptive fields; calculating the value of the second output variable (o.sub.010) by adding a product of d.sub.r2c and k.sub.r1c to the current value of the second output variable (o.sub.010); loading a next kernel element k.sub.r2c of the filter of the line r; calculating the value of the first output variable (o.sub.000) by adding a product of d.sub.r2c and k.sub.r2c to the current value of the first output variable (o.sub.000), and calculating the value of the second output variable (o.sub.010) by adding a product of d.sub.r3c and k.sub.r2c to the current value of the second output variable (o.sub.010).
2. The method according to claim 1, wherein: the filter and the input data have a plurality of channels, and a further loop is executed for each channel across the loop over the lines.
3. The method according to claim 1, wherein: the kernel elements have been read out line-by-line and the read-out lines are stored in a row as a sequence, and the loading the kernel elements is performed from the sequence.
4. The method according to claim 2, wherein: the method is performed by a data processing system, the data processing system executes two MAC instructions simultaneously, and two loops across the lines are executed for two channels simultaneously.
5. The method according to claim 1, wherein the filter comprises at least one 2×2 or 3×3 kernel.
6. The method according to claim 1, wherein the filter is a filter of a convolutional neural network.
7. The method according to claim 6, wherein the method is used to operate the convolutional neural network.
8. The method according to claim 1, wherein a computer program comprises instructions which, when the computer program is executed by a computer, prompt the computer to carry out the method.
9. A non-transitory machine-readable storage medium on which the computer program according to

claim 8 is stored.

10. A device configured to carry out the method according to claim 1.